

SIMATS ENGINEERING
ASSESSMENT TOOLS

CO5AT2

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

Design Critique Session

Code of Conduct:

I V. Smylka (192372115) (Name / Reg No) certify that this submission is my original work and adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V. Smylka
Signature of the student:

CO5: ASSESSMENT TOOL 1

V. Smykka
192372115
CSA1301

designed for $a^n b^n$ fails for $a^n b^n c^n$

$$L = \{a^n b^n \mid n \geq 1\}$$

- Reading every a and pushing one stack symbol.
- Reading every b and popping one stack symbol.
- Accepting when all a 's have been matched with b 's.

For ex:

aaabbb

xxx $\rightarrow$ xx $\rightarrow$ x $\rightarrow$ empty

This correctly checks that the number of a 's equal the number of b 's.

Why does it fail for $a^n b^n c^n$?

For

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

The machine has to verify 3 equal quantities:

$$\#a = \#b = \#c$$

A single-stack PDA cannot generally do this.

If the PDA pops its stack while reading the b 's, the information number of a 's is gone when it reaches the c 's.

How can be it improved?

For $a^n b^n c^n$, use a Turing Machine.

ATM can repeatedly:

$a \rightarrow x$

$b \rightarrow y$

$c \rightarrow z$

Conclusion:- $\{a^n b^n c^n \mid n \geq 1\}$ is not context-free.

$\therefore$ A standard one-stack PDA cannot recognize this language.
Improvement is to use a Turing machine or an equivalent

2. CFG to Deterministic PDA

A student claims that the CFG

$$E \rightarrow E^+ T / T$$

$$T \rightarrow T^+ F / F$$

$$F \rightarrow (E) / id$$

can be directly converted into a Deterministic PDA. Critically evaluate this statement.

The student's statement is not completely correct.

A standard conversion from a context-free grammar to a PDA produces a

Non-deterministic PDA (NPDA)

For example, when E is on top of the stack, the PDA has two possible

$$E \rightarrow E^+ T \quad \text{or} \quad E \rightarrow T$$

Thus, the PDA may have to choose between multiple transitions.

Similarly,

$$T \rightarrow T^+ F / F$$

also provides alternatives.

Therefore, the direct CFG-to-PDA construction is non-deterministic.

Why is this important?

A DPDA cannot arbitrarily choose between productions. Its next move based on the current state, input symbol, and stack symbol.

Correction

The correct statement is:

Every CFG can be converted directly into an NPDA, but into a DPDA is not guaranteed.

Conclusion.

CFG $\rightarrow$ NPDA is direct, but CFG $\rightarrow$ DPDA is not.

Alternative Turing Machine programming Techniques

Checking-off Symbols

In this technique, the TM marks symbols that have been already processed.

For example:

11111

After processing: xxxxx

For subtraction, the machine can mark one symbol from each number, and repeatedly compare them.

For ex: 11111 # 111

The machine can match the symbols one by one and mark them.

2. Subroutines:

A complex operation can be divided into smaller operations.

For ex:

Main program $\rightarrow$ Find First number $\rightarrow$ Find Second number $\rightarrow$ perform

conclusion:

Checking-off symbols, marking symbols, and subroutines are useful alternative tracks.

4. PDA Accepting strings with Equal Numbers of a and b

Design a PDA for $L = \{w \mid w \in \{a,b\}^* \text{ and } n_a(w) = n_b(w)\}$

The language contains all strings over $\{a,b\}$ having an equal number of a's and b's.

A PDA can use its stack to keep track of unmatched symbols. It can use x to represent an unmatched a and y to represent an unmatched b .

When an opposite symbol is encountered, the PDA can pop the corresponding symbol from the stack.

Stack symbol.

At the end, the PDA enters a final state only when there are no unmatched symbols.

If the PDA accepts by entering a final state, it can be designed as:

Input symbols $\rightarrow$ process a and b $\rightarrow$ Cancel matching symbols $\rightarrow$ No unmatched symbols $\rightarrow$ final state $\rightarrow$ Accept.

Empty string

Since $\epsilon \in (a+b)^*$ and $n_a(\epsilon) = n_b(\epsilon) = 0$,

The PDA must therefore account for ϵ if the definition includes it.

Conclusion:

The important requirement is that PDA must enter the final state only if the complete input has been processed and the number of a's equals the number of b's.

5. Comparison of FA, PDA and Turing Machine

Finite Automaton

A finite automaton has no additional memory apart from its finite state set.

It can recognize regular languages such as:

$$L = \{a^*b^*\}$$

but cannot recognize: $L = \{a^n b^n \mid n \geq 1\}$ because it cannot remember the number of a's.

Pushdown automaton

A PDA has a stack and is more powerful than an FA.

It can recognize: $\{a^n b^n \mid n \geq 1\}$ and is useful for:

- Balanced parentheses
- Expression parsing
- Nested structures

Turing Machine: A Turing Machine has an unbounded tape.

SIMATS ENGINEERING ASSESSMENT TOOLS

CRITERIA FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100

[Signature]

List x is ahead of y

Start with 1.

71, 42 = 1010